

Project Documentation - UCAB: Full-Stack Cab Booking & Ride-Hailing Platform

1. Introduction

Project Title: UCAB - Full-Stack Cab Booking & Ride-Hailing Platform

Project Contributors: Niharika Devi Guttula, Madiki Devi, Mahipala Veera Venkata Manikanta, Naveen Kumar Kondepudi, and Konatham Sreevanth

Niharika Devi Guttula - Role: Team Lead & Full-Stack Developer

Niharika Devi Guttula - Contribution: Architected system, designed database schemas, developed JWT authentication, and managed API integration.

Madiki Devi - Role: Frontend Developer & UI Designer

Madiki Devi - Contribution: Designed responsive React dashboards for Users/Drivers and styled the application components.

Mahipala Veera Venkata Manikanta - Role: Backend Developer & Database Engineer

Mahipala Veera Venkata Manikanta - Contribution: Implemented Express REST APIs for ride tracking and support; configured MongoDB Memory Server.

Naveen Kumar Kondepudi - Role: Full-stack Developer and Tester

Naveen Kumar Kondepudi - Contribution: Verified API endpoints, conducted system integration tests, handled error middleware, and resolved issues.

Konatham Sreevanth - Role: Full-stack Developer and Documentation Specialist

Konatham Sreevanth - Contribution: Contributed to full-stack development, conducted system testing, validated workflows, and compiled documentation.

2. Project Overview

Purpose: The purpose of this project is to automate the cab booking and ride-hailing process. It replaces manual coordination with centralized records, automated fare estimation, driver status tracking, and request handling.

Features: User and driver JWT authentication; ride requests; driver status updates (accepting, starting, completing rides); admin dashboard for driver approval and ticket management; simulated payments; support ticketing system.

3. Architecture

Frontend: React.js framework (Vite) utilizing custom CSS for modern, responsive layouts, Axios for API communications, and custom contexts for global state management.

Backend: Node.js and Express.js REST API using JWT-based token verification, role-based protection middlewares, and ticket handling controllers.

Database: MongoDB database utilizing Mongoose ODM schemas for User, Driver, Ride, Payment, and Support models. Includes automated seeding and in-memory server simulation.

4. Setup Instructions

Prerequisites: Node.js, internet connection, and a supported modern web browser such as Chrome or Edge.

Installation / Configuration: Clone the project; run 'npm run install-all' to install dependencies for frontend and backend; configure '.env' variables in the backend; run 'npm run dev' to launch concurrent development servers; test with default seeded accounts.

5. Application Components

User Interface: Responsive views for registration, login, user dashboard, driver dashboard, and admin dashboard.

Data Layer: Mongoose schemas for structured storage of users, drivers, rides, payments, and support ticket details.

Automation Layer: Express routing and state machine handlers processing ride request states from submission to completion.

Security Layer: JWT verification filters, bcryptjs password hashing, and role-based route guard verification.

6. Running the Application

Onboarding / User Flow: Log in as a user, enter pickup and drop locations, select vehicle type, request a ride, track status updates, and view payment completion.

Driver Flow: Log in as a driver, toggle availability, view and accept incoming ride requests, update ride states (Arrived, In Progress, Completed).

Admin Flow: Log in as admin, view registration requests, approve/reject new drivers, manage active support tickets, and review overall ride metrics.

7. Workflow Operations

Key Operations: User account creation; driver verification; ride state-machine updates; automatic fare estimation; simulated payments; support complaint resolution.

8. Authentication and Access

Security: JWT-based sessions verify client identity. Role protection ensures that driver registration reviews are admin-exclusive, ride-handling is driver-exclusive, and ride creation is user-exclusive.

9. User Interface

Interface: The application utilizes React forms, tables, and modal components for booking, request lists, driver registrations, and customer support tracking, offering dynamic dashboards for all roles.

10. Testing

Tests Performed: Verified registration validation; verified JWT authentication on restricted routes; simulated end-to-end ride flows (creation, assignment, updates); verified simulated payment updates; tested driver approval flows and ticket creation.

11. Demo Link

https://drive.google.com/file/d/1EUKk-PCnRqCHPiMmYS2czJ8_a0bRNhfy/view?usp=sharing

12. Known Issues

Limitations: By default, the backend depends on MongoDB Memory Server; database records will be reset on application restart. The current location is hardcoded and simulated using coordinates rather than live GPS tracking.

13. Future Enhancements

Enhancements: Integration with Google Maps API for path finding and live location updates; payment gateway integration (such as Stripe or Razorpay); real-time WebSocket connection for passenger-driver updates; mobile application with React Native.